notifications See all notifications


search...
0
enadeaud

- Manage slots
- Settings
- **Logout**

Have a problem?

- Profile
- Projets
- E-learning
- Forum
- Companies
- Meta
- Shop

Menu
My projects Holy Graph List projects Available Cursus
Your projects
Call Me Maybe Codexion Fly-in

Scale for project Codexion

You should evaluate 1 student in this team

Git repository
git@vogsphere.42mulhouse.fr

Introduction

- Remain polite, courteous, respectful and constructive
throughout the evaluation process. The well-being of the community
depends on it.

- Identify with the person (or the group) evaluated the eventual
dysfunctions of the work. Take the time to discuss
and debate the problems you have identified.

- You must consider that there might be some difference in how your
peers might have understood the project's instructions and the
scope of its functionalities. Always keep an open mind and grade
him/her as honestly as possible. The pedagogy is valid only and
only if peer-evaluation is conducted seriously.

Guidelines

- Only grade the work that is in the student or group's
GiT repository.

- Double-check that the GiT repository belongs to the student
or the group. Ensure that the work is for the relevant project

and also check that "git clone" is used in an empty folder.

- Check carefully that no malicious aliases was used to fool you
and make you evaluate something other than the content of the
official repository.

- To avoid any surprises, carefully check that both the evaluating
and the evaluated students have reviewed the possible scripts used
to facilitate the grading.

- If the evaluating student has not completed that particular
project yet, it is mandatory for this student to read the
entire subject prior to starting the defence.

- Use the flags available on this scale to signal an empty repository,
non-functioning program, a norm error, cheating etc. In these cases,
the grading is over and the final grade is 0 (or -42 in case of
cheating). However, with the exception of cheating, you are
encouraged to continue to discuss your work (even if you have not
finished it) in order to identify any issues that may have caused
this failure and avoid repeating the same mistake in the future.

- Remember that for the duration of the defence, no segfault,
no other unexpected, premature, uncontrolled or unexpected
termination of the program, else the final grade is 0. Use the
appropriate flag.
You should never have to edit any file except the configuration file if it exists.
If you want to edit a file, take the time to explicit the reasons with the
evaluated student and make sure both of you are okay with this.

- You must also verify the absence of memory leaks. Any memory allocated on the heap must
be properly freed before the end of execution.
You are allowed to use any of the different tools available on the computer, such as
leaks, valgrind, or e_fence. In case of memory leaks, tick the appropriate flag.

- Since this is a concurrency project, you must also check for data races and
deadlocks, which are not always visible by simply reading the logs. Use the
thread-specific tooling available on the computer, for example:
- valgrind --tool=helgrind ./codexion ...
- valgrind --tool=drd ./codexion ...
- recompiling the project with -fsanitize=thread and running it.
If any of these tools reports a data race or a lock-ordering deadlock, tick
the appropriate flag. Do not simply trust the evaluated student's explanation
that the code is thread-safe.

Attachments

subject.pdf

Preliminaries

Basics

- This project must be compiling with:
-Wall -Wextra -Werror -pthread
without any warnings.

- Any compilation error, segmentation fault, data race, deadlock, interleaved log line, or memory leak means 0 to the project.
- The program must take exactly the following arguments, in order: number_of_coders, time_to_burnout, time_to_compile, time_to_debug, time_to_refactor, number_of_compiles_required, dongle_cooldown, scheduler.
- Each coder must be represented by a thread (using pthread_create).
- The scheduler must be either "fifo" or "edf".
- On some slow hardware, precise timing may vary slightly; discuss borderline cases honestly before counting them as failures.
- Verify compilation: make
- The binary must be named "codexion".

○ Yes ◉ No

Global

- Check if there is any global mutable variable used to manage the shared resources (dongles), the scheduling policy, or the logging.
- If you find such a global state, the evaluation stops here.
  You can go on and check the code, but do not grade the exercises.

○ Yes ◉ No

README.md file

Does the repository contain a README.md file at its root, and does it include all of the following?

- The first line is italicized and formatted exactly as: *This project has been created as part of the 42 curriculum by <login1>[, <login2> , <login3>[...]]*.
- A "Description" section explaining the project's purpose and providing a brief overview.
- An "Instructions" section with relevant details about compilation, installation, and/or execution.
- A "Resources" section listing references (documentation, tutorials, etc.) and explaining how AI was used, specifying for which tasks and which parts of the project.
- A "Blocking cases handled" section describing all the concurrency issues addressed in the solution.
- A "Thread synchronization mechanisms" section explaining the specific threading primitives used.

○ Yes ◉ No

Codexion

Codexion testing — Easy

- Do not test with more than 200 coders.
- Do not test with timing values under 60ms.
- number_of_compiles_required must be at least 1: with 0, every coder has already "compiled at least 0 times" at startup and the simulation may stop immediately.
- The first case below must burn out (it checks burnout detection); the other cases must run without any burnout and complete all required compiles.

Reference behaviour (use these as benchmark cases). Arguments are, in order:
number_of_coders time_to_burnout time_to_compile time_to_debug
time_to_refactor number_of_compiles_required dongle_cooldown scheduler

- ./codexion 1 800 200 200 200 10 0 fifo A single coder has a single dongle but needs two to compile. The coder can therefore never compile and MUST burn out, around timestamp 800.
- ./codexion 5 2000 200 200 200 10 0 fifo time_to_burnout (2000) is well above one compile+debug+refactor cycle (200+200+200 = 600), so no coder should burn out; the program must stop on its own once every coder has compiled at least 10 times.
- ./codexion 5 2000 200 200 200 7 0 edf Same, with the edf scheduler: no burnout, stops after 7 compiles each.

As there is no reference binary, these cases and their expected behaviour
ARE the benchmark. Because of OS scheduling, run each case several times and
expect the correct behaviour in the large majority of runs (allow a small
timing tolerance on borderline cases before counting them as failures).

○ Yes ◉ No

Codexion testing — Less easy

- Validate burnout edge cases.
- Verify correct logging and timing tolerances.
- Check no dongle duplication, correct state transitions.

Reference behaviour (benchmark cases):

- ./codexion 5 500 200 200 200 10 0 fifo Infeasible by design: one compile+debug+refactor cycle takes at least 200+200+200 = 600 ms, which is longer than time_to_burnout (500), so a coder cannot start its next compile in time and MUST burn out (around timestamp 500). The "burned out" line must be the LAST line printed and must appear within 10 ms of the deadline.
- On a feasible run (for example the 5 2000 ... case from the Easy section), check that every "is compiling" line is immediately preceded by exactly two "has taken a dongle" lines for the same coder, and that no two log lines are ever interleaved on the same line.
- Confirm the same dongle is never held by two coders at the same time (no dongle duplication).

○ Yes ◉ No

Codexion testing — Medium

- Verify cooldown behavior.
- Test scheduler differences (edf vs fifo).
- Validate refactoring timing and log serialization.

Reference behaviour (benchmark cases):

- ./codexion 5 3000 200 200 200 10 400 fifo Cooldown check: after a coder releases a dongle, that dongle must not be taken again until 400 ms have elapsed. By reading the timestamps, verify that no "has taken a dongle" line for a given dongle occurs within 400 ms of its release. time_to_burnout (3000) is generous, so no coder should burn out and the run should complete.
- ./codexion 5 3000 200 200 200 10 800 fifo then ... edf Run the exact same parameters once with fifo and once with edf. The large cooldown (800) forces coders to queue for the dongles. Compare the order in which the waiting coders are granted the dongles: under edf, a coder closer to its burnout deadline may be served ahead of one that requested earlier, so the grant order can differ from the strict arrival order seen with fifo.
- Verify each coder spends close to time_to_compile / time_to_debug / time_to_refactor in each phase (within timing tolerance), and that the output stays correctly serialized even under heavy contention.

○ Yes ◉ No

Codexion recode — FIFO to LIFO

- Ask the evaluated student to turn the "fifo" scheduler into a "lifo" one: when several coders are waiting for the same dongle, it must now be granted to the coder whose request arrived LAST instead of first.
- This change is observable at runtime, unlike a tie on EDF deadlines. Run the program once before the change and once after, with the SAME contention-heavy parameters (for example: ./codexion 5 3000 200 200 200 10 800 fifo), and compare the order in which the waiting coders acquire the dongles in the logs.
- The recode is validated if the grant order under "lifo" is the reverse of the arrival order (the most recent requester is served first), while the rest of the program keeps working: no coder duplicates a dongle, logs stay serialized, and, with feasible parameters, no coder burns out.
- Also read the code to confirm the change is made in the scheduler/queue logic itself and not faked at the log level.

○ Yes ◉ No

Ratings

Don’t forget to check the flag corresponding to the defense

Ok Outstanding project
Empty work Incomplete work Invalid compilation Norme Cheat Crash Concerning situation Leaks Forbidden function Can’t support / explain code

Conclusion

Flag
◉ Ok ○ Incomplete work ○ Invalid compilation ○ Norme ○ Cheat ○ Crash ○ Outstanding project ○ Concerning situation ○ Leaks ○ Forbidden Function ○ Can’t support / explain code ○ Empty work

Leave a comment on this evaluation (2048 chars max)

Finish evaluation

API General Terms of UseDeclaration on the use of cookiesPrivacy policyGeneral term of use of the siteRules of procedureTerms of use for video surveillanceLegal notices

x

Cancel Send

x

Flash modal content (raw)

Close